Platform Framework - Technical Specifications

📦 Overview

This document provides technical specifications, code structure, configuration examples, and usage guidelines for all 8 platform starters without the actual implementation code.

🏗️ 1. PLATFORM-PARENT-BOM

Purpose

Central dependency management and version alignment across all microservices.

Maven Coordinates

```
<groupId>com.company.insurance.platform</groupId>
<artifactId>platform-parent-bom</artifactId>
<version>1.0.0</version>
<packaging>pom</packaging>
```

Code Structure

```
platform-parent-bom/
├── pom.xml                    # Parent POM with dependency management
├── README.md                  # Usage documentation
└── CHANGELOG.md               # Version history
```

Key Dependencies Managed

Dependency	Version	Purpose
Spring Boot	3.4.1	Core framework
Spring Cloud	2023.0.3	Cloud patterns
Resilience4j	2.2.0	Resilience patterns
Kafka	3.8.0	Event streaming
Avro	1.12.0	Schema evolution

Dependency	Version	Purpose
JWT	0.12.6	Token handling
Oracle JDBC	23.5.0.24.07	Database driver
Micrometer	1.13.5	Metrics
OpenTelemetry	1.42.1	Distributed tracing

Configuration Example

```
<!-- In microservice pom.xml -->
<parent>
    <groupId>com.company.insurance.platform</groupId>
    <artifactId>platform-parent-bom</artifactId>
    <version>1.0.0</version>
</parent>

<dependencies>
    <!-- No version tags needed - managed by parent -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
</dependencies>
```

Usage Guidelines

- ✅ Always use as parent POM for all microservices
- ✅ Never override managed dependency versions
- ✅ Update parent version for all services together
- ❌ Don't add business dependencies to parent BOM

📝 2. PLATFORM-LOGGING-STARTER

Purpose

Standardized logging with correlation ID propagation, MDC support, and structured JSON logging.

Maven Coordinates

```
<groupId>com.company.insurance.platform</groupId>
<artifactId>platform-logging-starter</artifactId>
<version>1.0.0</version>
```

Code Structure

```
platform-logging-starter/
├── src/main/java/
│   └── com/company/insurance/platform/logging/
│       ├── config/
│       │   └── LoggingAutoConfiguration.java      # Auto-config
│       ├── filter/
│       │   └── CorrelationIdFilter.java           # HTTP filter
│       ├── mdc/
│       │   ├── MdcTaskDecorator.java              # Async support
│       │   └── MdcKafkaConsumerInterceptor.java   # Kafka support
│       ├── interceptor/
│       │   └── RequestLoggingInterceptor.java     # Request logging
│       └── util/
│           └── CorrelationIdUtil.java             # Utility methods
├── src/main/resources/
│   ├── META-INF/
│   │   └── spring.factories                       # Auto-config registration
│   └── logback-spring.xml                         # Logback config
└── pom.xml
```

Key Components

Component	Responsibility
CorrelationIdFilter	Generate/extract correlation ID from HTTP headers
CorrelationIdUtil	Manage correlation ID in MDC
MdcTaskDecorator	Propagate MDC to async threads
MdcKafkaConsumerInterceptor	Propagate correlation ID via Kafka headers
RequestLoggingInterceptor	Log HTTP requests with method, path, latency

Configuration Example

```yaml
# application.yml
logging:
  level:
    root: INFO
    com.company.insurance: DEBUG
  pattern:
    console: "%d{yyyy-MM-dd HH:mm:ss} - %msg%n"

# Logback automatically configured for JSON logging
# Correlation ID automatically added to all logs
```

Usage Guidelines

```java
// Correlation ID is automatically available
@RestController
public class PolicyController {
```

```java
    @GetMapping("/{id}")
    public Policy getPolicy(@PathVariable String id) {
        // Correlation ID automatically in MDC
        String correlationId = CorrelationIdUtil.get();
        log.info("Fetching policy: id={}, correlationId={}", id, correlationId);
        return policyService.getPolicy(id);
    }
}

// For async operations
@Async
public void processAsync() {
    // Correlation ID automatically propagated
    log.info("Processing async task");
}

// For Kafka consumers
@KafkaListener(topics = "policy.issued")
public void handleEvent(PolicyIssuedEvent event) {
    // Correlation ID automatically extracted from Kafka headers
    log.info("Received event: {}", event);
}
```

🌐 3. PLATFORM-WEB-STARTER

Purpose

Standardize incoming HTTP handling with global exception handling, validation, and OpenAPI configuration.

Maven Coordinates

```maven
<groupId>com.company.insurance.platform</groupId>
<artifactId>platform-web-starter</artifactId>
<version>1.0.0</version>
```

Code Structure

```
platform-web-starter/
├── src/main/java/
│   └── com/company/insurance/platform/web/
│       ├── config/
│       │   ├── WebAutoConfiguration.java          # Auto-config
│       │   ├── ValidationConfig.java              # Bean validation
│       │   └── OpenApiConfig.java                 # Swagger/OpenAPI
│       ├── exception/
│       │   ├── GlobalExceptionHandler.java        # Exception handler
│       │   ├── BusinessException.java             # Business errors
│       │   ├── TechnicalException.java            # Technical errors
│       │   └── ResourceNotFoundException.java     # 404 errors
│       ├── model/
```

```
│       │   ├── ErrorResponse.java                # Error DTO
│       │   └── ValidationErrorResponse.java      # Validation DTO
│       └── advice/
│           └── ResponseBodyAdvice.java           # Response interceptor
└── pom.xml
```

Key Components

Component	Responsibility
GlobalExceptionHandler	Catch all exceptions, return standard error format
ValidationConfig	Configure Bean Validation (JSR-303)
OpenApiConfig	Configure Swagger UI and OpenAPI docs
ErrorResponse	Standard error response model
BusinessException	Domain-specific exceptions
TechnicalException	Infrastructure exceptions

Configuration Example

```yaml
# application.yml
springdoc:
  api-docs:
    path: /v3/api-docs
  swagger-ui:
    path: /swagger-ui.html
    enabled: true

spring:
  mvc:
    throw-exception-if-no-handler-found: true
  web:
    resources:
      add-mappings: false
```

Usage Guidelines

```java
// Automatic exception handling
@RestController
@RequestMapping("/api/policies")
public class PolicyController {

    // Validation automatically triggered
    @PostMapping
    public PolicyDto createPolicy(@Valid @RequestBody CreatePolicyRequest request) {
        // If validation fails, automatic 400 response with field errors
        return policyService.createPolicy(request);
    }
```

```
    // Business exceptions automatically handled
    @GetMapping("/{id}")
    public PolicyDto getPolicy(@PathVariable String id) {
        // Throws ResourceNotFoundException → automatic 404 response
        return policyService.getPolicy(id)
            .orElseThrow(() -> new ResourceNotFoundException("Policy", id));
    }
}

// Request DTO with validation
public class CreatePolicyRequest {
    @NotBlank(message = "Customer ID is required")
    private String customerId;

    @Positive(message = "Sum assured must be positive")
    private BigDecimal sumAssured;

    @Min(1) @Max(100)
    private Integer policyTerm;
}

// Standard error response format (automatic)
{
  "timestamp": "2024-02-16T10:00:00Z",
  "correlationId": "abc-123",
  "errorCode": "RESOURCE_NOT_FOUND",
  "message": "Policy not found with id: POL123",
  "path": "/api/policies/POL123"
}
```

🔌 4. PLATFORM-HTTP-CLIENT-STARTER

Purpose

Standardized outbound REST communication with resilience patterns, JWT propagation, and correlation ID propagation.

Maven Coordinates

```
<groupId>com.company.insurance.platform</groupId>
<artifactId>platform-http-client-starter</artifactId>
<version>1.0.0</version>
```

Code Structure

```
platform-http-client-starter/
├── src/main/java/
│   └── com/company/insurance/platform/httpclient/
│       ├── config/
│       │   ├── HttpClientAutoConfiguration.java   # Auto-config
│       │   ├── WebClientConfig.java               # WebClient setup
```

```
│       │   └── ResilienceConfig.java              # Resilience4j setup
│       ├── client/
│       │   ├── PlatformHttpClient.java           # Interface
│       │   └── PlatformHttpClientImpl.java       # Implementation
│       ├── interceptor/
│       │   ├── CorrelationIdInterceptor.java     # Correlation ID
│       │   ├── JwtPropagationInterceptor.java    # JWT token
│       │   └── MetricsInterceptor.java           # Metrics
│       ├── properties/
│       │   └── HttpClientProperties.java         # Configuration
│       └── exception/
│           └── HttpClientException.java          # HTTP errors
└── pom.xml
```

Key Components

Component	Responsibility
PlatformHttpClient	Unified interface for HTTP calls
WebClientConfig	Configure connection pool, timeouts
ResilienceConfig	Configure circuit breaker, retry, bulkhead
CorrelationIdInterceptor	Add correlation ID to outbound requests
JwtPropagationInterceptor	Propagate JWT token to downstream services
MetricsInterceptor	Record HTTP metrics (latency, errors)

Configuration Example

```yaml
# application.yml
platform:
  http-client:
    connection-pool:
      max-connections: 500
      max-connections-per-route: 100
      connection-timeout: 2s
      idle-timeout: 5m

    services:
      underwriting-service:
        base-url: http://underwriting-service:8080
        connect-timeout: 2s
        read-timeout: 5s
        circuit-breaker:
          enabled: true
          failure-rate-threshold: 50
          wait-duration-in-open-state: 60s
          sliding-window-size: 10
        retry:
          enabled: true
          max-attempts: 3
          wait-duration: 1s
          exponential-backoff-multiplier: 2.0
```

```yaml
      timeout:
        duration: 5s
      bulkhead:
        max-concurrent-calls: 10

    bre-service:
      base-url: http://bre-adapter:8080
      circuit-breaker:
        failure-rate-threshold: 60
      retry:
        max-attempts: 2
```

Usage Guidelines

```java
// Inject PlatformHttpClient
@Service
public class PolicyService {

    private final PlatformHttpClient httpClient;

    // Simple GET request
    public Mono<UnderwritingDecision> requestUnderwriting(String policyId) {
        return httpClient.get(
            "underwriting-service",
            "/api/underwriting/" + policyId,
            UnderwritingDecision.class
        );
        // Automatic: correlation ID, JWT, circuit breaker, retry, metrics
    }

    // POST request with body
    public Mono<BREResponse> evaluateRules(BRERequest request) {
        return httpClient.post(
            "bre-service",
            "/api/rules/evaluate",
            request,
            BREResponse.class
        );
    }

    // With custom headers
    public Mono<Response> customCall() {
        HttpHeaders headers = new HttpHeaders();
        headers.set("X-Custom-Header", "value");

        return httpClient.get(
            "underwriting-service",
            "/api/custom",
            headers,
            Response.class
        );
    }

    // With fallback
    public Mono<BREResponse> evaluateWithFallback(BRERequest request) {
        return httpClient.post("bre-service", "/api/rules", request, BREResponse.class)
            .onErrorResume(ex -> {
                log.error("BRE failed, using default rules", ex);
```

```
                return Mono.just(getDefaultRules());
            });
    }
}
```

📨 5. PLATFORM-KAFKA-STARTER

Purpose

Standardized Kafka messaging with event envelope, dead letter strategy, and observability.

Maven Coordinates

```
<groupId>com.company.insurance.platform</groupId>
<artifactId>platform-kafka-starter</artifactId>
<version>1.0.0</version>
```

Code Structure

```
platform-kafka-starter/
├── src/main/java/
│   └── com/company/insurance/platform/kafka/
│       ├── config/
│       │   ├── KafkaAutoConfiguration.java         # Auto-config
│       │   ├── KafkaProducerConfig.java            # Producer setup
│       │   └── KafkaConsumerConfig.java            # Consumer setup
│       ├── model/
│       │   └── EventEnvelope.java                  # Generic wrapper
│       ├── producer/
│       │   └── EventPublisher.java                 # Event publishing
│       ├── consumer/
│       │   └── DeadLetterPublisher.java            # DLT handling
│       ├── interceptor/
│       │   ├── CorrelationIdProducerInterceptor.java
│       │   └── CorrelationIdConsumerInterceptor.java
│       └── properties/
│           └── KafkaProperties.java                # Configuration
└── pom.xml
```

Key Components

Component	Responsibility
EventEnvelope	Generic event wrapper with metadata
EventPublisher	Publish events to Kafka topics
DeadLetterPublisher	Handle failed messages (DLT)

Component	Responsibility
CorrelationIdProducerInterceptor	Add correlation ID to Kafka headers
CorrelationIdConsumerInterceptor	Extract correlation ID from headers

Event Envelope Structure

```
EventEnvelope<T> {
    String eventId;           // Unique event ID (UUID)
    String eventType;         // Event type (e.g., "PolicyIssued")
    Instant timestamp;        // Event timestamp
    String source;            // Source service name
    String correlationId;     // Correlation ID for tracing
    T payload;                // Business payload (generic)
    String version;           // Event schema version
    Map<String, String> metadata;  // Additional metadata
}
```

Configuration Example

```yaml
# application.yml
spring:
  kafka:
    bootstrap-servers: kafka-broker-1:9092,kafka-broker-2:9092
    producer:
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      value-serializer: org.springframework.kafka.support.serializer.JsonSerializer
      acks: all
      retries: 3
      properties:
        linger.ms: 10
        batch.size: 16384
        compression.type: snappy
    consumer:
      key-deserializer: org.apache.kafka.common.serialization.StringDeserializer
      value-deserializer: org.springframework.kafka.support.serializer.JsonDeserializer
      group-id: ${spring.application.name}
      auto-offset-reset: earliest
      enable-auto-commit: false
      properties:
        spring.json.trusted.packages: "*"
    listener:
      ack-mode: manual
      concurrency: 3

platform:
  kafka:
    dead-letter:
      enabled: true
      max-retry-attempts: 3
      backoff-initial-interval: 1000
      backoff-multiplier: 2.0
      backoff-max-interval: 10000
```

Usage Guidelines

```java
// Publishing events
@Service
public class PolicyService {

    private final EventPublisher eventPublisher;

    public void issuePolicy(Policy policy) {
        // Business logic
        policy.setStatus(PolicyStatus.ISSUED);
        policyRepository.save(policy);

        // Publish event
        PolicyIssuedEvent event = PolicyIssuedEvent.builder()
            .policyId(policy.getId())
            .customerId(policy.getCustomerId())
            .productCode(policy.getProductCode())
            .sumAssured(policy.getSumAssured())
            .build();

        eventPublisher.publish(
            "policy.issued",           // Topic
            policy.getId(),            // Key (for partitioning)
            "PolicyIssued",            // Event type
            event                      // Payload
        );
        // Automatic: eventId, timestamp, source, correlationId, version
    }
}

// Consuming events
@Component
public class PolicyEventConsumer {

    @KafkaListener(topics = "policy.issued", groupId = "underwriting-service")
    public void handlePolicyIssued(
            @Payload EventEnvelope<PolicyIssuedEvent> envelope,
            Acknowledgment acknowledgment) {

        log.info("Received PolicyIssued: eventId={}, policyId={}",
                envelope.getEventId(),
                envelope.getPayload().getPolicyId());

        try {
            // Process event
            underwritingService.processNewPolicy(envelope.getPayload());

            // Manual acknowledgment
            acknowledgment.acknowledge();
        } catch (Exception ex) {
            log.error("Failed to process event", ex);
            // Dead letter handling automatic after max retries
        }
    }
}
```

🔐 6. PLATFORM-SECURITY-STARTER

Purpose

Standardized authentication and authorization with JWT validation and method security.

Maven Coordinates

```
<groupId>com.company.insurance.platform</groupId>
<artifactId>platform-security-starter</artifactId>
<version>1.0.0</version>
```

Code Structure

```
platform-security-starter/
├── src/main/java/
│   └── com/company/insurance/platform/security/
│       ├── config/
│       │   ├── SecurityAutoConfiguration.java      # Auto-config
│       │   ├── JwtSecurityConfig.java              # Security chain
│       │   └── MethodSecurityConfig.java           # Method security
│       ├── filter/
│       │   └── JwtAuthenticationFilter.java        # JWT filter
│       ├── jwt/
│       │   ├── JwtTokenValidator.java              # Token validation
│       │   └── JwtTokenExtractor.java              # Token extraction
│       ├── util/
│       │   └── SecurityContextUtil.java            # Utility methods
│       ├── exception/
│       │   ├── JwtAuthenticationException.java     # Auth errors
│       │   └── SecurityExceptionHandler.java       # Error handler
│       └── properties/
│           └── SecurityProperties.java             # Configuration
└── pom.xml
```

Key Components

Component	Responsibility
JwtAuthenticationFilter	Extract and validate JWT from requests
JwtTokenValidator	Validate JWT signature, expiration, claims
JwtTokenExtractor	Extract JWT from Authorization header
SecurityContextUtil	Access current user, roles, permissions
MethodSecurityConfig	Enable @PreAuthorize, @Secured annotations

Configuration Example

```yaml
# application.yml
platform:
  security:
    jwt:
      issuer: apigee-gateway
      secret: ${JWT_SECRET}  # From environment/secrets manager
      expiration-ms: 3600000  # 1 hour
      header-name: Authorization
      header-prefix: "Bearer "

    public-endpoints:
      - /actuator/health
      - /actuator/info
      - /api/public/**
      - /swagger-ui/**
      - /v3/api-docs/**

    admin-endpoints:
      - /api/admin/**
      - /actuator/**

    cors:
      enabled: true
      allowed-origins:
        - http://localhost:3000
        - https://app.company.com
      allowed-methods:
        - GET
        - POST
        - PUT
        - DELETE
      allowed-headers:
        - "*"
      allow-credentials: true
```

Usage Guidelines

```java
// Method-level security
@RestController
@RequestMapping("/api/policies")
public class PolicyController {

    // Role-based access
    @PreAuthorize("hasRole('UNDERWRITER')")
    @PostMapping
    public PolicyDto createPolicy(@Valid @RequestBody CreatePolicyRequest request) {
        return policyService.createPolicy(request);
    }

    // Multiple roles
    @PreAuthorize("hasRole('UNDERWRITER') or hasRole('ADMIN')")
    @GetMapping("/{id}")
    public PolicyDto getPolicy(@PathVariable String id) {
        return policyService.getPolicy(id);
```

```java
    }

    // Admin only
    @Secured("ROLE_ADMIN")
    @DeleteMapping("/{id}")
    public void deletePolicy(@PathVariable String id) {
        policyService.deletePolicy(id);
    }

    // Custom expression
    @PreAuthorize("@customSecurityService.isPolicyOwner(#id)")
    @PutMapping("/{id}")
    public PolicyDto updatePolicy(@PathVariable String id, @RequestBody UpdateRequest request) {
        return policyService.updatePolicy(id, request);
    }
}

// Access security context
@Service
public class PolicyService {

    public void createPolicy(CreatePolicyRequest request) {
        // Get current user
        String username = SecurityContextUtil.getCurrentUsername()
            .orElseThrow(() -> new UnauthorizedException("Not authenticated"));

        // Check role
        if (!SecurityContextUtil.hasRole("UNDERWRITER")) {
            throw new AccessDeniedException("Insufficient permissions");
        }

        // Business logic
        Policy policy = new Policy();
        policy.setCreatedBy(username);
        policyRepository.save(policy);
    }
}
```

📊 7. PLATFORM-OBSERVABILITY-STARTER

Purpose

Uniform observability with metrics, distributed tracing, and health indicators.

Maven Coordinates

```pom.properties
<groupId>com.company.insurance.platform</groupId>
<artifactId>platform-observability-starter</artifactId>
<version>1.0.0</version>
```

Code Structure

```
platform-observability-starter/
├── src/main/java/
│   └── com/company/insurance/platform/observability/
│       ├── config/
│       │   ├── ObservabilityAutoConfiguration.java # Auto-config
│       │   ├── MicrometerConfig.java               # Metrics setup
│       │   └── TracingConfig.java                  # Tracing setup
│       ├── metrics/
│       │   ├── CustomMetrics.java                  # Business metrics
│       │   └── MetricsNamingConvention.java        # Naming standards
│       ├── health/
│       │   ├── DatabaseHealthIndicator.java        # DB health
│       │   ├── KafkaHealthIndicator.java           # Kafka health
│       │   └── RedisHealthIndicator.java           # Redis health
│       └── properties/
│           └── ObservabilityProperties.java        # Configuration
└── pom.xml
```

Key Components

Component	Responsibility
CustomMetrics	Record business metrics (policies issued, claims)
MicrometerConfig	Configure metrics registry, common tags
TracingConfig	Configure OpenTelemetry, sampling
DatabaseHealthIndicator	Check database connectivity
KafkaHealthIndicator	Check Kafka broker connectivity
RedisHealthIndicator	Check Redis connectivity

Configuration Example

```yaml
# application.yml
management:
  endpoints:
    web:
      exposure:
        include: health,info,metrics,prometheus
  endpoint:
    health:
      show-details: always
      show-components: always
  metrics:
    tags:
      application: ${spring.application.name}
      environment: ${ENV:dev}
    distribution:
      percentiles-histogram:
        http.server.requests: true
      slo:
        http.server.requests: 100ms,200ms,500ms,1s,2s,5s
```

```yaml
platform:
  observability:
    metrics:
      enabled: true
      prefix: insurance
      include-host-tag: true
      include-application-tag: true

    tracing:
      enabled: true
      sampling-probability: 1.0  # 100% sampling (adjust for prod)
      exporter-endpoint: http://jaeger:4318/v1/traces
      service-name: ${spring.application.name}

    health:
      show-details: true
      enable-database-check: true
      enable-kafka-check: true
      enable-redis-check: true
```

Usage Guidelines

```java
// Record business metrics
@Service
public class PolicyService {

    private final CustomMetrics metrics;

    public Policy issuePolicy(CreatePolicyRequest request) {
        long startTime = System.currentTimeMillis();

        try {
            // Business logic
            Policy policy = createPolicy(request);

            // Record success metrics
            metrics.recordPolicyIssued(policy.getProductCode(), "WEB");
            metrics.recordOperation("policy_issuance", "success");

            long duration = System.currentTimeMillis() - startTime;
            metrics.recordOperationDuration("policy_issuance", duration);

            return policy;
        } catch (Exception ex) {
            // Record failure metrics
            metrics.recordOperation("policy_issuance", "failure");
            throw ex;
        }
    }
}

// Distributed tracing
@Service
public class PolicyService {

    private final Tracer tracer;
```

```java
    public Policy issuePolicy(CreatePolicyRequest request) {
        // Create span
        Span span = tracer.spanBuilder("issuePolicy")
            .setAttribute("policyId", request.getPolicyId())
            .setAttribute("customerId", request.getCustomerId())
            .startSpan();

        try (Scope scope = span.makeCurrent()) {
            // Business logic - automatically traced
            return createPolicy(request);
        } finally {
            span.end();
        }
    }
}

// Health checks (automatic)
// Access: GET /actuator/health
{
  "status": "UP",
  "components": {
    "database": {
      "status": "UP",
      "details": {
        "database": "Oracle",
        "status": "Connected"
      }
    },
    "kafka": {
      "status": "UP",
      "details": {
        "brokers": 3,
        "status": "Connected"
      }
    },
    "redis": {
      "status": "UP",
      "details": {
        "status": "Connected"
      }
    }
  }
}
```

🗄️ 8. PLATFORM-CACHING-STARTER

Purpose

Standardized caching infrastructure with Redis, TTL configuration, and service isolation.

Maven Coordinates

```publication_info
<groupId>com.company.insurance.platform</groupId>
<artifactId>platform-caching-starter</artifactId>
```

```
<version>1.0.0</version>
```

Code Structure

```
platform-caching-starter/
├── src/main/java/
│   └── com/company/insurance/platform/caching/
│       ├── config/
│       │   ├── CachingAutoConfiguration.java       # Auto-config
│       │   ├── RedisCacheConfig.java               # Redis setup
│       │   └── CaffeineCacheConfig.java            # Caffeine setup
│       ├── properties/
│       │   └── CachingProperties.java              # Configuration
│       └── metrics/
│           └── CacheMetrics.java                   # Cache metrics
└── pom.xml
```

Key Components

Component	Responsibility
RedisCacheConfig	Configure Redis as distributed cache
CaffeineCacheConfig	Configure Caffeine as in-memory cache
CachingProperties	Per-cache TTL and configuration
CacheMetrics	Record cache hit/miss metrics

Configuration Example

```yaml
# application.yml
platform:
  caching:
    type: REDIS  # or CAFFEINE or HYBRID

    redis:
      host: ${REDIS_HOST:localhost}
      port: ${REDIS_PORT:6379}
      password: ${REDIS_PASSWORD:}
      database: 0
      timeout: 2s
      ssl: false

    caffeine:
      initial-capacity: 100
      maximum-size: 1000
      expire-after-write: 10m
      expire-after-access: 5m

    caches:
      product-config:
        ttl: 1h
```

```yaml
      allow-null-values: false
      key-prefix: product

    master-data:
      ttl: 24h
      allow-null-values: false
      key-prefix: master

    token-introspection:
      ttl: 5m
      allow-null-values: false
      key-prefix: token

    idempotency-keys:
      ttl: 24h
      allow-null-values: false
      key-prefix: idempotency

    customer-profile:
      ttl: 15m
      allow-null-values: false
      key-prefix: customer
```

Usage Guidelines

```java
// Cacheable methods
@Service
public class ProductService {

    // Cache read
    @Cacheable(value = "product-config", key = "#productCode")
    public ProductConfig getProductConfig(String productCode) {
        log.info("Fetching from database: {}", productCode);
        return productRepository.findByCode(productCode)
            .orElseThrow(() -> new ResourceNotFoundException("Product", productCode));
    }

    // Cache eviction
    @CacheEvict(value = "product-config", key = "#productCode")
    public void updateProductConfig(String productCode, ProductConfig config) {
        log.info("Updating and evicting cache: {}", productCode);
        productRepository.save(config);
    }

    // Clear all cache
    @CacheEvict(value = "product-config", allEntries = true)
    public void clearAllProductCache() {
        log.info("Clearing all product cache");
    }

    // Conditional caching
    @Cacheable(value = "product-config", key = "#productCode",
               condition = "#productCode != null",
               unless = "#result == null")
    public ProductConfig getProductConfigConditional(String productCode) {
        return productRepository.findByCode(productCode).orElse(null);
    }
}
```

```java
// Master data caching
@Service
public class MasterDataService {

    @Cacheable(value = "master-data", key = "'country-codes'")
    public List<CountryCode> getCountryCodes() {
        log.info("Fetching country codes from database");
        return masterDataRepository.findAllCountryCodes();
    }

    @Cacheable(value = "master-data", key = "'occupation-codes'")
    public List<OccupationCode> getOccupationCodes() {
        log.info("Fetching occupation codes from database");
        return masterDataRepository.findAllOccupationCodes();
    }
}

// Idempotency key caching
@Service
public class IdempotencyService {

    @Cacheable(value = "idempotency-keys", key = "#requestId")
    public String checkIdempotency(String requestId) {
        // Check if request already processed
        return idempotencyRepository.findByRequestId(requestId)
            .map(Idempotency::getResponseId)
            .orElse(null);
    }

    @CachePut(value = "idempotency-keys", key = "#requestId")
    public String storeIdempotency(String requestId, String responseId) {
        Idempotency idempotency = new Idempotency(requestId, responseId);
        idempotencyRepository.save(idempotency);
        return responseId;
    }
}
```

📋 SUMMARY TABLE

Starter	Purpose	Key Features	Auto-Configured
parent-bom	Dependency management	Version alignment, plugin config	N/A
logging	Structured logging	Correlation ID, MDC, JSON logs	✅
web	HTTP handling	Exception handling, validation, OpenAPI	✅
http-client	Outbound REST	Resilience, JWT propagation, metrics	✅
kafka	Event streaming	Event envelope, DLT, correlation ID	✅
security	Authentication	JWT validation, method security	✅
observability	Monitoring	Metrics, tracing, health checks	✅

Starter	Purpose	Key Features	Auto-Configured
caching	Performance	Redis/Caffeine, TTL, metrics	✅

🎯 USAGE PATTERNS

Pattern 1: REST API Microservice

```
<dependencies>
    <dependency>
        <groupId>com.company.insurance.platform</groupId>
        <artifactId>platform-logging-starter</artifactId>
    </dependency>
    <dependency>
        <groupId>com.company.insurance.platform</groupId>
        <artifactId>platform-web-starter</artifactId>
    </dependency>
    <dependency>
        <groupId>com.company.insurance.platform</groupId>
        <artifactId>platform-security-starter</artifactId>
    </dependency>
    <dependency>
        <groupId>com.company.insurance.platform</groupId>
        <artifactId>platform-observability-starter</artifactId>
    </dependency>
</dependencies>
```

Pattern 2: Event-Driven Microservice

```
<dependencies>
    <dependency>
        <groupId>com.company.insurance.platform</groupId>
        <artifactId>platform-logging-starter</artifactId>
    </dependency>
    <dependency>
        <groupId>com.company.insurance.platform</groupId>
        <artifactId>platform-kafka-starter</artifactId>
    </dependency>
    <dependency>
        <groupId>com.company.insurance.platform</groupId>
        <artifactId>platform-observability-starter</artifactId>
    </dependency>
</dependencies>
```

Pattern 3: Full-Stack Microservice

```
<dependencies>
    <!-- All 7 starters -->
    <dependency>
```

```
        <groupId>com.company.insurance.platform</groupId>
        <artifactId>platform-logging-starter</artifactId>
    </dependency>
    <dependency>
        <groupId>com.company.insurance.platform</groupId>
        <artifactId>platform-web-starter</artifactId>
    </dependency>
    <dependency>
        <groupId>com.company.insurance.platform</groupId>
        <artifactId>platform-http-client-starter</artifactId>
    </dependency>
    <dependency>
        <groupId>com.company.insurance.platform</groupId>
        <artifactId>platform-kafka-starter</artifactId>
    </dependency>
    <dependency>
        <groupId>com.company.insurance.platform</groupId>
        <artifactId>platform-security-starter</artifactId>
    </dependency>
    <dependency>
        <groupId>com.company.insurance.platform</groupId>
        <artifactId>platform-observability-starter</artifactId>
    </dependency>
    <dependency>
        <groupId>com.company.insurance.platform</groupId>
        <artifactId>platform-caching-starter</artifactId>
    </dependency>
</dependencies>
```

🚀 GETTING STARTED

Step 1: Create Microservice

```
# Create new Spring Boot project
spring init --dependencies=web \
  --group-id=com.company.insurance \
  --artifact-id=policy-service \
  --name=PolicyService \
  policy-service
```

Step 2: Add Platform Parent

```
<parent>
    <groupId>com.company.insurance.platform</groupId>
    <artifactId>platform-parent-bom</artifactId>
    <version>1.0.0</version>
</parent>
```

Step 3: Add Required Starters

```xml
<dependencies>
    <dependency>
        <groupId>com.company.insurance.platform</groupId>
        <artifactId>platform-logging-starter</artifactId>
    </dependency>
    <dependency>
        <groupId>com.company.insurance.platform</groupId>
        <artifactId>platform-web-starter</artifactId>
    </dependency>
    <!-- Add other starters as needed -->
</dependencies>
```

Step 4: Configure Application

```yaml
# application.yml
spring:
  application:
    name: policy-service

platform:
  security:
    jwt:
      secret: ${JWT_SECRET}
  observability:
    metrics:
      enabled: true
```

Step 5: Build and Run

```bash
mvn clean install
mvn spring-boot:run
```